



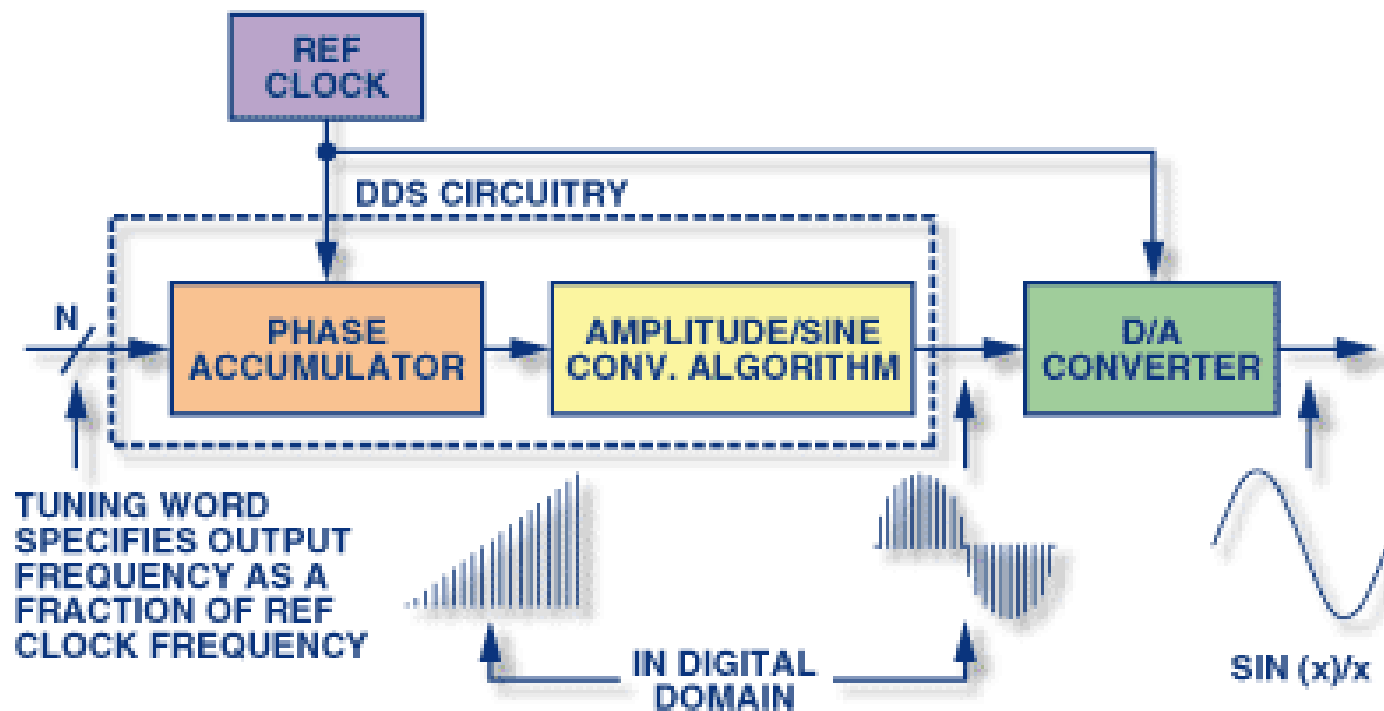
Nội dung trình bày:

1. Tổng quan hệ thống phát sóng số (DDS)
2. Thiết kế lõi IP phần cứng
3. Kiến trúc SoC & Giao tiếp Avalon-MM
4. Tích hợp phần mềm trên Nios II
5. So sánh với các bài báo
6. Mô phỏng và Nạp KIT DE2



1. Tổng quan về hệ thống phát sóng số (DDS)

DDS (Direct Digital Synthesizer): Là kỹ thuật tạo ra các dạng sóng tương tự (Sin, Cos, Tam giác, Vuông) từ một hệ thống mạch kỹ thuật số.





1. Tổng quan về hệ thống phát sóng số (DDS)

Bộ cộng dồn pha (Phase Accumulator):

- Chức năng: Tích lũy giá trị bước nhảy pha sau mỗi chu kỳ clock.
- Quyết định: Tần số của tín hiệu ngõ ra.

Bộ chuyển đổi Pha sang Biên độ (Phase-to-Amplitude Converter):

- Chức năng: Ánh xạ giá trị pha từ bộ cộng dồn thành biên độ sóng tương ứng.
- Phương pháp thực hiện: Bảng tra bộ nhớ (LUT) hoặc Thuật toán CORDIC.



1. Tổng quan về hệ thống phát sóng số (DDS)

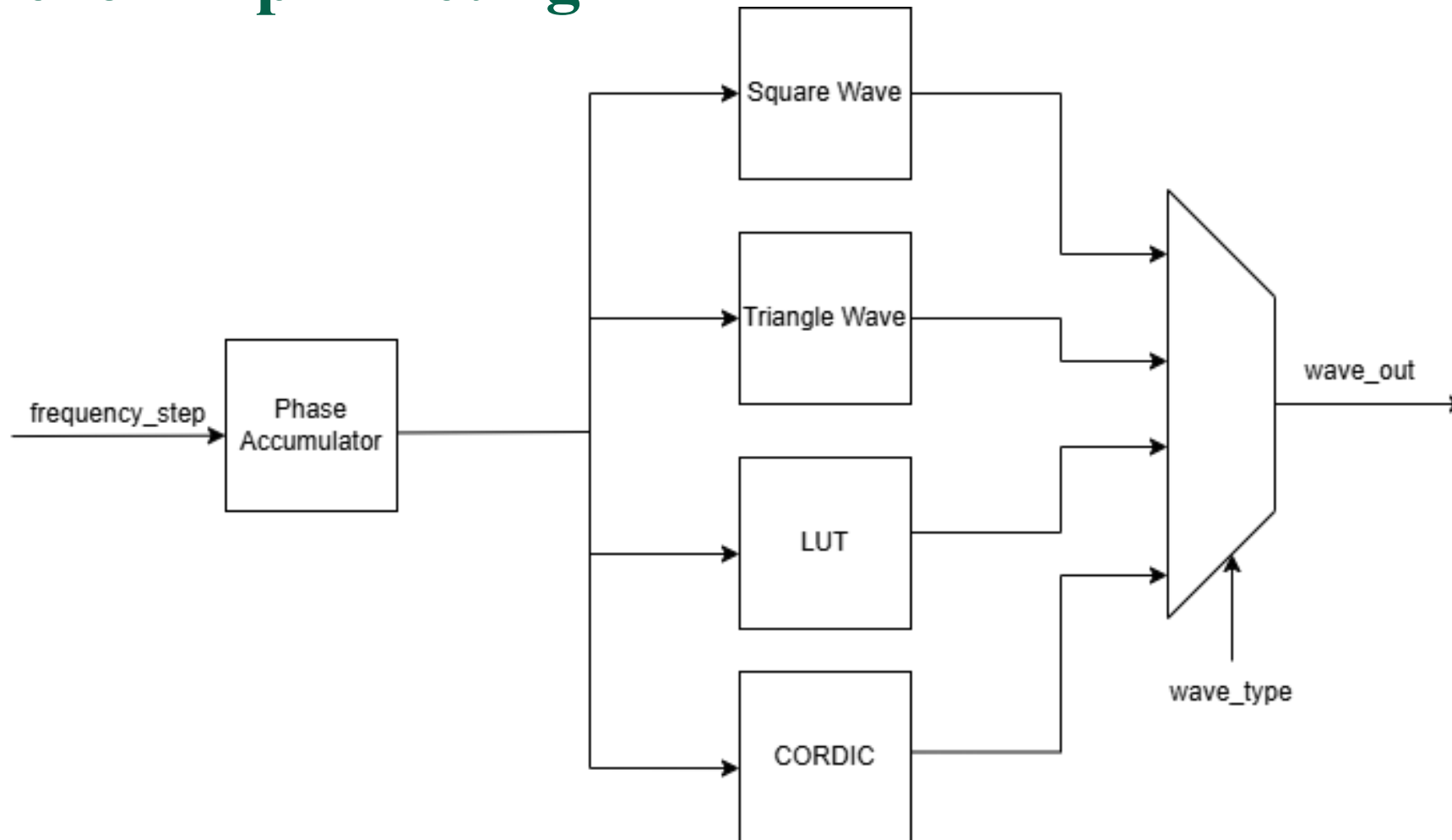
Công thức tính tần số từ bước nhảy pha:

$$f_{out} = \frac{M * f_{clk}}{2^N}$$

- f_{out} : Tần số của tín hiệu sóng ngõ ra mà hệ thống cần tổng hợp.
- M : Bước nhảy pha (Phase Increment)
- f_{clk} : Tần số xung nhịp hoạt động của toàn hệ thống.
- 2^N : Độ phân giải pha tối đa, đại diện cho tổng số điểm pha rời rạc trong một chu kỳ sóng hoàn chỉnh.



2. Thiết kế lõi IP phần cứng



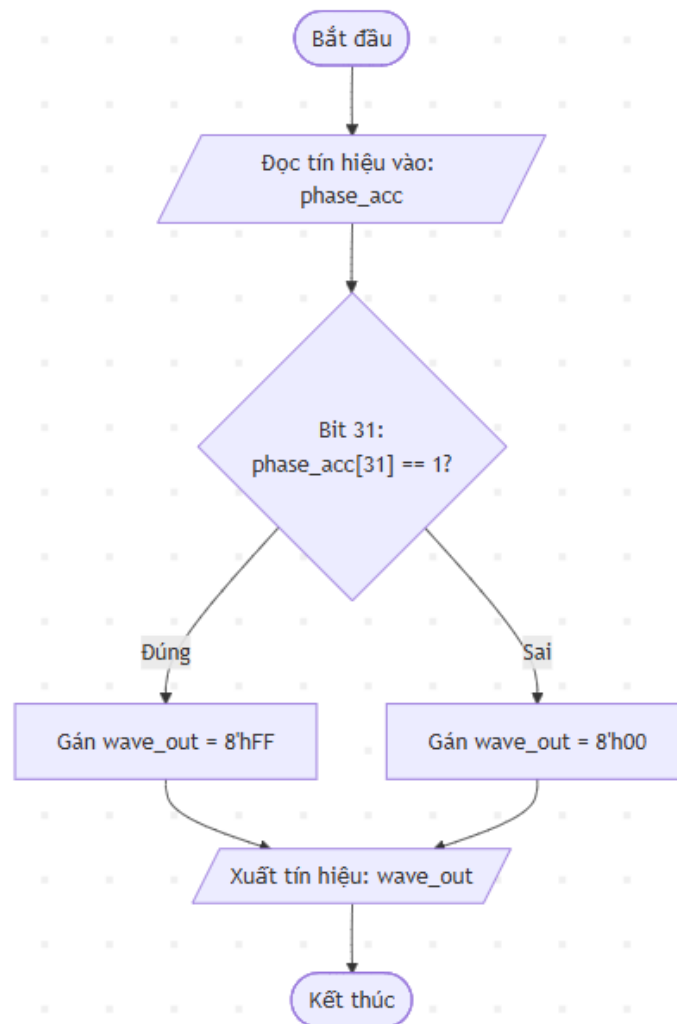


2. Thiết kế lõi IP phần cứng

2.1. IP tạo sóng vuông

Nguyên lý hoạt động:

- Lấy giá trị bit cao bộ cộng pha.
- Thực hiện gán giá trị cho ngõ ra phụ thuộc vào giá trị bit cao ngõ vào.



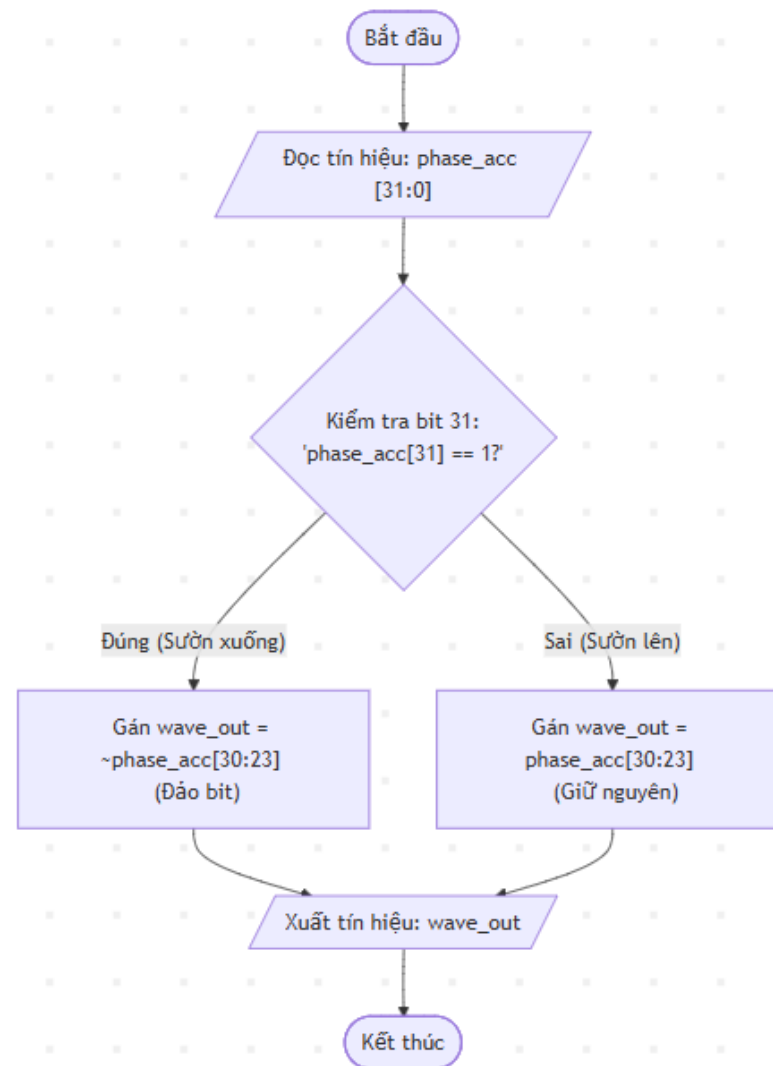


2. Thiết kế lõi IP phần cứng

2.2. IP tạo sóng tam giác

Nguyên lý hoạt động:

- Lấy giá trị bit cao bộ cộng pha.
- Gán giá trị ngõ ra bằng hoặc đảo của 8 bit cao của bộ cộng pha.





2. Thiết kế lõi IP phần cứng

2.3. IP tạo sóng Sin/Cosin bằng phương pháp Bảng tra (LUT)

Nguyên lý thiết kế:

- Sử dụng tài nguyên bộ nhớ nội của FPGA (Block RAM) để cấu hình một bảng ROM lưu trữ 256 giá trị biên độ mẫu của một chu kỳ sóng Sin.
- Dữ liệu biên độ (8-bit) được nạp sẵn từ tệp tin `sine.txt` trong quá trình khởi tạo hệ thống.



2. Thiết kế lõi IP phần cứng

2.3. IP tạo sóng Sin/Cosin bằng phương pháp Bảng tra (LUT)

Cơ chế thực thi thuật toán:

- Ánh xạ địa chỉ: Trích xuất 8-bit trọng số cao nhất của bộ cộng dồn pha (`phase_acc[31:24]`) làm chỉ số địa chỉ để truy vấn bảng ROM.
- Tạo sóng Sin: Giá trị biên độ được đọc trực tiếp từ ROM tại địa chỉ tương ứng: `sin_out <= sin_rom[index]`.
- Tạo sóng Cosin: Tận dụng tính chất lệch pha 90 độ trong lượng giác. Với bảng 256 mẫu, khoảng lệch này tương ứng với 64 đơn vị địa chỉ: `cos_out <= sin_rom[index + 64]`.



2. Thiết kế lõi IP phần cứng

2.3. IP tạo sóng Sin/Cosin bằng phương pháp Bảng tra (LUT)

Đánh giá kỹ thuật:

- Tốc độ: Thời gian đáp ứng cực nhanh, dữ liệu xuất hiện ngay sau khi có địa chỉ (độ trễ truy cập RAM thấp).
- Tài nguyên: Tiêu tốn tài nguyên Block RAM tỉ lệ thuận với độ phân giải và số lượng mẫu lưu trữ. Đây là cơ sở để so sánh hiệu quả với thuật toán CORDIC ở phần tiếp theo.

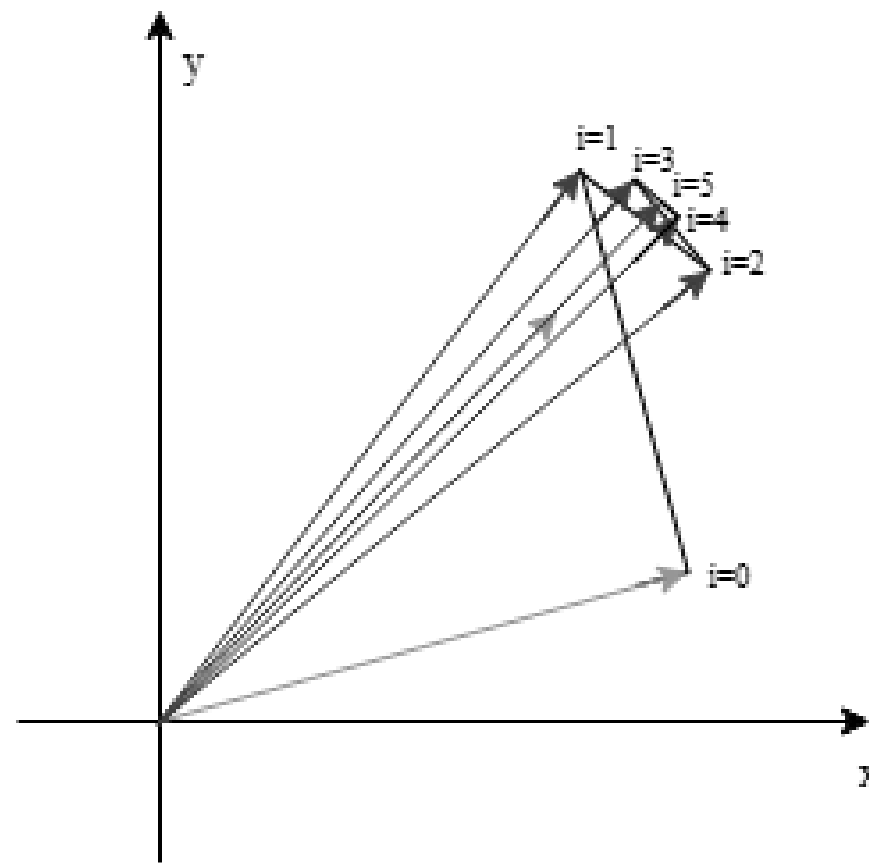


2. Thiết kế lõi IP phần cứng

2.4. IP tạo sóng Sin/Cosin bằng thuật toán CORDIC

Nguyên lý thiết kế:

- Sử dụng thuật toán xoay vector liên tiếp để tính toán giá trị Sin và Cos trực tiếp tại thời gian thực.
- Tối ưu hóa mức cổng logic: Loại bỏ hoàn toàn các bộ nhân, thay thế bằng các phép dịch bit và cộng/trừ.





2. Thiết kế lõi IP phần cứng

2.4. Lõi IP tạo sóng Sin/Cosin bằng thuật toán CORDIC

Nguyên lý thiết kế:

- Xử lý góc phần tư: Sử dụng 2-bit cao nhất của bộ cộng pha (`phase_acc[31:30]`) để đưa góc về phạm vi 0 - 90 độ trước khi tính toán.
- Kiến trúc Pipeline 8 tầng: Thuật toán được chia thành 8 giai đoạn xử lý nối tiếp. Tại mỗi tầng, vector được xoay một góc $\arctan(2^{-i})$.
- Hiệu chỉnh ngõ ra: Khôi phục dấu dựa trên góc phần tư ban đầu và thực hiện dịch trục biên độ (+128) để tạo tín hiệu không dấu 8-bit tương thích với hệ thống.



2. Thiết kế lõi IP phần cứng

2.4. Lõi IP tạo sóng Sin/Cosin bằng thuật toán CORDIC

Đánh giá kỹ thuật:

- Tài nguyên: Hoàn toàn không sử dụng bộ nhớ RAM, giúp tiết kiệm tài nguyên Block RAM cho các thành phần khác của SoC.
- Hiệu năng: Kiến trúc Pipeline cho phép hệ thống đạt tần số hoạt động (F_{max}) cao và xuất ra một mẫu dữ liệu mới trên mỗi chu kỳ xung nhịp.



2. Thiết kế lõi IP phần cứng

2.5. Tính linh hoạt trong thiết kế IP Core

Thiết kế tích hợp tham số tùy biến **USE_CORDIC** cho phép tối ưu hóa tài nguyên dựa trên đặc thù chip FPGA: lựa chọn LUT để đạt tốc độ phản hồi cực nhanh (tốn RAM) hoặc thuật toán CORDIC để tiết kiệm tuyệt đối bộ nhớ bằng tài nguyên logic. Sự linh hoạt này giúp lõi IP dễ dàng thích ứng với mọi kịch bản triển khai.



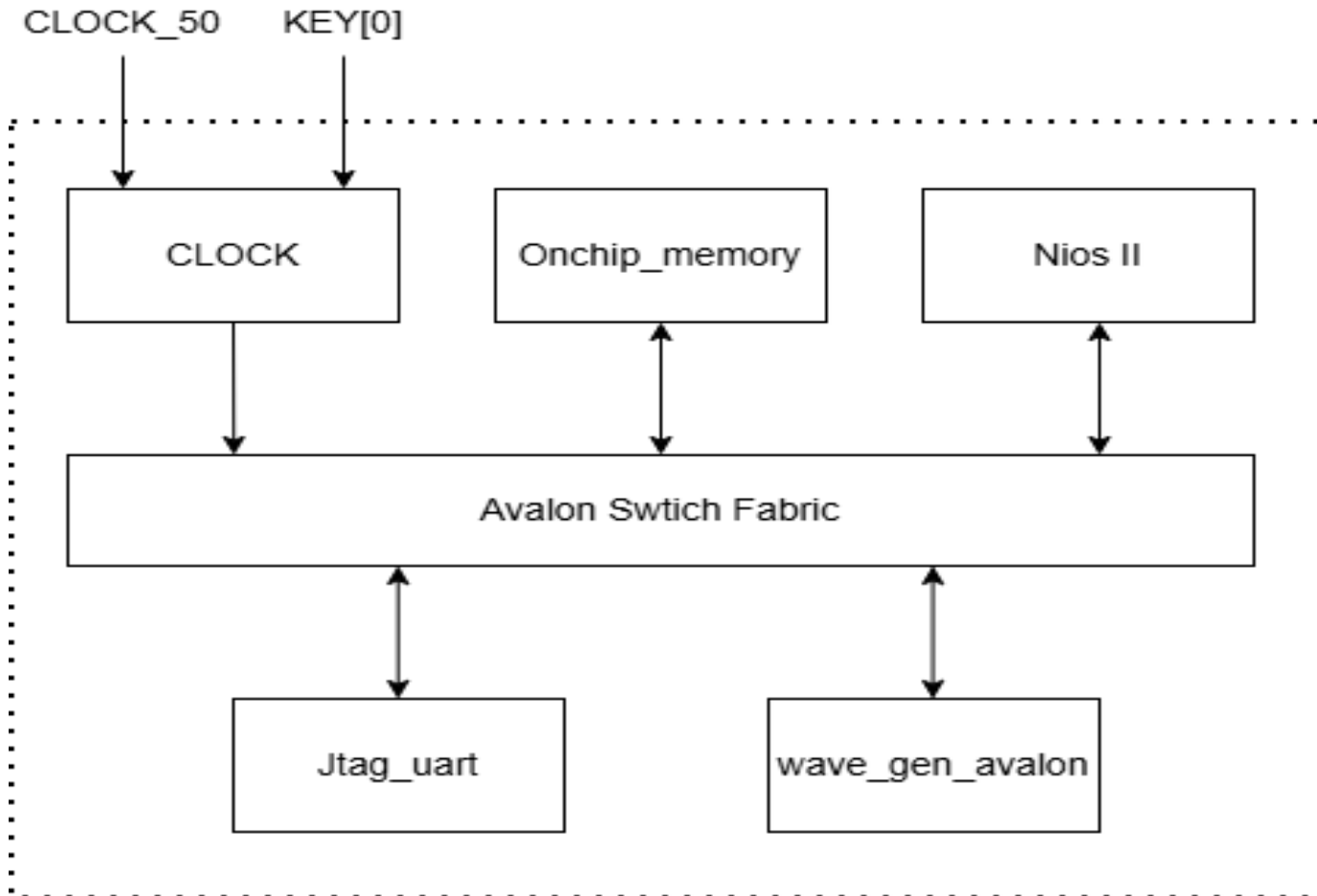
2. Thiết kế lõi IP phần cứng

2.5. Tính linh hoạt trong thiết kế IP Core

Tiêu chí	Bảng tra (LUT)	Thuật toán CORDIC
Tài nguyên RAM	Tốn Block RAM (tỉ lệ với mẫu)	Không sử dụng RAM (0 bit)
Tài nguyên Logic	Rất thấp	Trung bình (Adder/Shifter)
Latency	1 - 2 Clock cycles	8 Clock cycles (Pipeline)
Độ chính xác	Phụ thuộc mật độ bảng tra	Rất cao (độ phân giải bit)



3. Kiến trúc SoC & Giao tiếp Avalon-MM





3. Kiến trúc SoC & Giao tiếp Avalon-MM

iClk, iReset_n

Cấp xung nhịp (50MHz) và tín hiệu khởi tạo hệ thống đồng bộ.

Select, Write, Read, Address

Điều phối việc truy cập và giải mã địa chỉ thanh ghi từ Nios II.

iData / oReadData [31:0]

Bus dữ liệu 2 chiều: Nhận lệnh cấu hình (Write) và phản hồi trạng thái (Read).

oData [7:0] (Conduit)

Tín hiệu sóng ngõ ra xuất trực tiếp tới chân GPIO vật lý.

```
module wave_generator_avalon #(
    parameter USE_CORDIC = 1
) (
    input iClk, iReset_n,
    input iChipSelect_n,
    input iWrite_n, iRead_n,
    input [1:0] iAddress,
    input [31:0] iData,
    output reg [31:0] oReadData,
    output [7:0] oData // Conduit Out
);
```



3. Kiến trúc SoC & Giao tiếp Avalon-MM

Khai báo thanh ghi nội

Sử dụng **reg** 32-bit và 2-bit để lưu giữ cấu hình từ CPU, làm trung gian cung cấp dữ liệu cho lõi xử lý sóng.

Trạng thái Khởi tạo (Reset)

Khi có tín hiệu **iReset_n**, hệ thống tự động thiết lập tần số mặc định và dạng sóng cơ bản để đảm bảo ổn định.

Ánh xạ Địa chỉ

Sử dụng **iAddress** làm tín hiệu chọn luồng. Dữ liệu từ bus **iData** sẽ được chọt vào đúng thanh ghi khi Master cấp lệnh Ghi.

```
reg [31:0] reg_frequency_step;
reg [1:0] reg_wave_type;

always @(posedge iClk or negedge iReset_n) begin
    if (!iReset_n) begin
        reg_frequency_step <= 32'd10000;
        reg_wave_type <= 2'd0;
    end else if (!iChipSelect_n && !iWrite_n) begin
        case (iAddress)
            2'b00: reg_frequency_step <= iData;
            2'b01: reg_wave_type <= iData[1:0];
        endcase
    end
end
```



3. Kiến trúc SoC & Giao tiếp Avalon-MM

Mạch Tổ hợp

Khác với logic Ghi, logic Đọc được thiết kế dạng **Mạch tổ hợp** giúp tối ưu hóa tốc độ truy xuất.

Gộp Bit

Vì Bus dữ liệu rộng 32-bit, thanh ghi **reg_wave_type** (2-bit) được gộp thêm 30 bit '0' để đảm bảo tính toàn vẹn dữ liệu khi truyền về CPU.

Trạng thái An toàn

Khi không có lệnh Đọc, ngõ ra **oReadData** luôn được gán bằng 0. Điều này giúp tránh xung đột dữ liệu trên Bus Avalon và tiết kiệm năng lượng.

```
always @(*) begin
    if (!iChipSelect_n && !iRead_n) begin
        case (iAddress)
            2'b00: oReadData = reg_frequency_step;
            2'b01: oReadData = {30'd0, reg_wave_type};
            default: oReadData = 32'd0;
        endcase
    end else begin
        oReadData = 32'd0;
    end
end
```



3. Kiến trúc SoC & Giao tiếp Avalon-MM

Truyền Tham số

Tham số **USE_CORDIC** từ lớp vỏ Avalon được truyền thẳng xuống lõi Core bên dưới, cho phép người dùng tinh chỉnh kiến trúc trực tiếp từ giao diện Qsys.

Bơm dữ liệu & Xuất tín hiệu

Các thanh ghi nội được nối trực tiếp vào ngõ vào của Lõi. Đồng thời, ngõ ra **wave_out** được nối xuyên thấu ra **oData** để thoát khỏi hệ thống Bus.

```
wave_generator_core #(
    .USE_CORDIC(USE_CORDIC)
) u_wave_core (
    .clk(iClk),
    .reset(~iReset_n), // Đảo cực tín hiệu
    .frequency_step(reg_frequency_step),
    .wave_type(reg_wave_type),
    .wave_out(oData) // Nối ra Conduit
);

endmodule
```



3. Kiến trúc SoC & Giao tiếp Avalon-MM

Tích hợp lõi IP DDS vào kiến trúc SoC trên công cụ Qsys.

Qsys - system.qsys (D:\SoC\Doan_Trong\DoAn\system.qsys)

File Edit System View Tools Help

Component Library

Project: wave_gen_avalon

Library: Config-Bypass App Example

System Contents

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		clk_0	Clock Source	clk	clk_0				
☒		clk_in	Clock Input	reset					
☒		clk_in_reset	Reset Input						
☒		clk	Clock Output						
☒		clk_reset	Reset Output						
☒		nios2_qsys_0	Nios II Processor						
☒		clk	Clock Input		clk_0				
☒		reset_n	Reset Input		[clk]				
☒		data_master	Avalon Memory Mapped Master		[clk]				
☒		instruction_master	Avalon Memory Mapped Master		[clk]				
☒		flag_debug_module_re	Reset Output		[clk]				
☒		flag_debug_module_re	Avalon Memory Mapped Slave		[clk]	# 0x0001_0800	0x0001_0fff		
☒		custom_instruction_m	Custom Instruction Master		[clk]				
☒		onchip_memory2_0	On-Chip Memory (RAM or ROM)						
☒		clk1	Clock Input		clk_0				
☒		s1	Avalon Memory Mapped Slave		[clk1]	# 0x0000_8000	0x0000_ffff		
☒		reset1	Reset Input						
☒		flag_uart_0	JTAG UART						
☒		clk	Clock Input		clk_0				
☒		reset	Reset Input		[clk]				
☒		avalon_flag_slave	Avalon Memory Mapped Slave		[clk]	# 0x0001_1010	0x0001_101f		
☒		wave_gen_avalon_0	wave_gen_avalon						
☒		avalon_slave_0	Avalon Memory Mapped Slave		[clock_sink]	# 0x0001_1000	0x0001_100f		
☒		clock_sink	Clock Input		clk_0				
☒		reset_sink	Reset Input		[clock_sink]				
☒		conduit_end	Conduit	wave_gen_avalon_0_con...					

Messages

Description

Path

0 Errors, 0 Warnings



3. Kiến trúc SoC & Giao tiếp Avalon-MM

wave_gen_avalon - wave_gen_avalon_0

wave_gen_avalon
wave_gen_avalon

Documentation

Block Diagram

☐ Show signals

```
graph LR
    wave_gen_avalon_0 -- avalon_slave_0 --> wave_gen_avalon
    style wave_gen_avalon_0 fill:#fff,stroke:#000,stroke-width:1px
    style wave_gen_avalon fill:#fff,stroke:#000,stroke-width:1px
```

Parameters

USE_CORDIC: 1

Cancel

Finish

Component Editor - wave_gen_avalon_hw.tcl

File Templates

Component Type

Files

Parameters

Signals

Interfaces

About Signals

Name	Interface	Signal Type	Width	Direction
iWrite_n	avalon_slave_0	write_n	1	input
iRead_n	avalon_slave_0	read_n	1	input
iAddress	avalon_slave_0	address	2	input
iData	avalon_slave_0	writedata	32	input
iChipSelect_n	avalon_slave_0	chipselect_n	1	input
oReadData	avalon_slave_0	readdata	32	output
iClk	clock_sink	clk	1	input
iReset_n	reset_sink	reset_n	1	input
oData	conduit_end	export	8	output



4. Tích hợp phần mềm trên Nios II

```
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include "system.h"
#include "io.h"

// Định nghĩa Trạng thái Sóng
#define WAVE_SQUARE 0
#define WAVE_TRIANGLE 1
#define WAVE_SINE 2
#define WAVE_COSINE 3

// Offset Thanh ghi Avalon
#define REG_FREQ_OFFSET 0
#define REG_TYPE_OFFSET 1

// Thông số Kỹ thuật DDS
#define SYSTEM_CLOCK_MHZ 50.0
#define PHASE_ACC_MAX 4294967296.0
```

Thư viện HAL

system.h cung cấp các macro địa chỉ vật lý (Base Address) được sinh ra tự động từ Qsys. **io.h** cung cấp các lệnh đọc/ghi cấp thấp để tương tác với Avalon Bus.

Ánh xạ Thanh ghi

Các hằng số **OFFSET 0** và **OFFSET 1** tương ứng chính xác với địa chỉ 2'b00 và 2'b01 đã thiết kế trong khối phần cứng Avalon Wrapper.

Hằng số Tính toán DDS

Sử dụng **50.0 MHz** làm xung nhịp hệ thống. Đặc biệt, hằng số **4294967296.0** chính là 2^{32} - đại diện cho độ phân giải tối đa của Bộ cộng dồn pha (Phase Accumulator) 32-bit.



4. Tích hợp phần mềm trên Nios II

```
void wave_set_frequency(float freq_MHz) {
    double step_calc = (freq_MHz * PHASE_ACC_MAX) /
        SYSTEM_CLOCK_MHZ;
    uint32_t step = (uint32_t)step_calc;
    IOWR(WAVE_GEN_AVALON_0_BASE,
        REG_FREQ_OFFSET, step);
}

void wave_set_type(uint8_t type) {
    IOWR(WAVE_GEN_AVALON_0_BASE,
        REG_TYPE_OFFSET, (type & 0x03));
}
```

Hàm Cấu hình Tần số

Vì xử lý tính toán bước nhảy pha (step) theo công thức DDS. Phép tính sử dụng kiểu **double** để triệt tiêu sai số làm tròn khi chia cho tần số hệ thống, sau đó mới ép về kiểu **uint32_t** để nạp vào thanh ghi.

Ghi dữ liệu xuống IP Core

Sử dụng lệnh **IOWR** để truyền dữ liệu qua Avalon-MM. Địa chỉ ghi được xác định bằng Base Address của khối IP kết hợp với **OFFSET 0** (Thanh ghi tần số) hoặc **OFFSET 1** (Thanh ghi chọn dạng sóng).

Bảo vệ phần cứng

Phép toán **AND 0x03** bảo vệ IP Core khỏi các giá trị lựa chọn sóng không hợp lệ.



4. Tích hợp phần mềm trên Nios II

```
#include main() {
    printf("Bat dau chay SignalTap Test...\n");
    wave_set_frequency(0.5);
    while(1) {
        printf("Phat: Song Vuong (500 KHz)\n");
        wave_set_type(WAVE_SQUARE);
        usleep(2000000);

        printf("Phat: Song Tam Giac (500 KHz)\n");
        wave_set_type(WAVE_TRIANGLE);
        usleep(2000000);

        printf("Phat: Song Sin(500 KHz)\n");
        wave_set_type(WAVE_SINE);
        usleep(2000000);

        printf("Phat: Song Cosine(500 KHz)\n");
        wave_set_type(WAVE_COSINE);
        usleep(2000000);
    }
    return 0;
}
```

Khởi tạo hệ thống

Ngay sau khi khởi động, chương trình thiết lập tần số hoạt động mặc định là **500 KHz**. Lệnh này gửi thông số bước nhảy pha trực tiếp xuống thanh ghi phân cứng.

Vòng lặp vô tận

Sử dụng cấu trúc while(1) để đảm bảo hệ thống duy trì việc phát tín hiệu liên tục, phục vụ quá trình đo đạc và quan sát dạng sóng trong thời gian dài.

Chuyển đổi dạng sóng tuần tự

Hệ thống tự động luân chuyển giữa các dạng sóng (Vuông, Tam giác, Sin, Cos). Mỗi trạng thái được duy trì trong **2 giây** nhờ hàm usleep, cho phép bộ SignalTap II bắt mẫu tín hiệu ổn định.



5. So sánh với các bài báo

5.1. Báo cáo tài nguyên

CORDIC

Revision Name	wave_generator
Top-level Entity Name	wave_generator_core
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	1,281 / 33,216 (4 %)
Total combinational functions	882 / 33,216 (3 %)
Dedicated logic registers	992 / 33,216 (3 %)
Total registers	992
Total pins	44 / 475 (9 %)
Total virtual pins	0
Total memory bits	64 / 483,840 (< 1 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)

LUT

Revision Name	wave_generator
Top-level Entity Name	wave_generator_core
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	802 / 33,216 (2 %)
Total combinational functions	438 / 33,216 (1 %)
Dedicated logic registers	688 / 33,216 (2 %)
Total registers	688
Total pins	44 / 475 (9 %)
Total virtual pins	0
Total memory bits	2,048 / 483,840 (< 1 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)



5. So sánh với các bài báo

5.1. Báo cáo tài nguyên

Quy đổi Bram LUT:

- $2048 / 18432 \approx 0.111$ Bram
- LUT: $0.111 \times 800 \approx 89$
- FF: $0.111 \times 32 = 3.5$

Tổng tài nguyên:

$$802 \text{ (LUT)} + 688 \text{ (FF)} + 89 + 3.5 = 1582.5$$



5. So sánh với các bài báo

5.1. Báo cáo tài nguyên

Quy đổi Bram CORDIC:

- $64 / 18432 \approx 0.003$ Bram
- LUT: $0.003 \times 800 = 2.4$
- FF: $0.003 \times 32 = 0.096$

Tổng tài nguyên:

$$1281 \text{ (LUT)} + 992 \text{ (FF)} + 2.4 + 0.096 = 2275.496$$



5. So sánh với các bài báo

5.2. Tần số tối đa

CORDIC

Slow Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	208.68 MHz	208.68 MHz	my_system_clock	

208.68MHz

LUT

Slow Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	195.85 MHz	195.85 MHz	my_system_clock	

195.85MHz



5. So sánh với các bài báo

5.2. So sánh

Bài báo 1: Pipelined CORDIC Design on FPGA for a Digital Sine and Cosine Waves Generator - Esteban O. Garcia, Rene Cumplido, Miguel Arias

TABLE I
SPEED OPTIMIZATION RESULTS

Parameter	Used	%
Number of Slices	1104	57
Number of Flip Flops	615	16
Numer of 4 input LUTs	1748	45
Number of bonded IOBs	43	24
Number of GCLKs	1	12
Maximum Frequency	154.69MHz	

TABLE II
AREA OPTIMIZATION RESULTS

Parameter	Used	%
Number of Slices	1075	55
Number of Flip Flops	570	14
Numer of 4 input LUTs	1737	44
Number of bonded IOBs	43	24
Number of GCLKs	1	12
Maximum Frequency	124.67MHz	



5. So sánh với các bài báo

5.2. So sánh

Tổng tài nguyên tối ưu tốc độ:

$$615 \text{ (FF)} + 1748 \text{ (LUT)} = 2363$$

Tổng tài nguyên tối ưu diện tích:

$$570 \text{ (FF)} + 1737 \text{ (LUT)} = 2307$$

=> Kết luận: Thiết kế của nhóm tối ưu hơn thiết kế trong bài báo cả về mặt diện tích ($2275 < 2307$) lẫn tốc độ ($208.68\text{MHz} > 154.69\text{MHz}$)



5. So sánh với các bài báo

5.2. So sánh

Bài báo 2: High-precision design of DDS Based on FPGA - Xiaochu Wang, Qiujun Mei

```
Flow Status                Successful - Wed Dec 28 22:22:04 2011
Quartus II Version         9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name              DDS
Top-level Entity Name     DDS
Family                     Cyclone III
Device                     EP3C5M164C7
Timing Models              Final
Met timing requirements    N/A
Total logic elements       2,200 / 5,136 ( 43 % )
  Total combinational functions 1,506 / 5,136 ( 29 % )
  Dedicated logic registers   2,160 / 5,136 ( 42 % )
Total registers            2160
Total pins                 98 / 107 ( 92 % )
Total virtual pins         0
Total memory bits          103 / 423,936 ( < 1 % )
Embedded Multiplier 9-bit elements 0 / 46 ( 0 % )
Total PLLs                 1 / 2 ( 50 % )
```



5. So sánh với các bài báo

5.2. So sánh

Quy đổi Bram:

- $103 / 18432 \approx 0.005$ Bram
- LUT: $0.005 \times 800 = 4$
- FF: $0.005 \times 32 = 0.16$

Tổng tài nguyên:

$$2200 (\text{LUT}) + 2160 (\text{FF}) + 4 + 0.16 = 4364.16$$

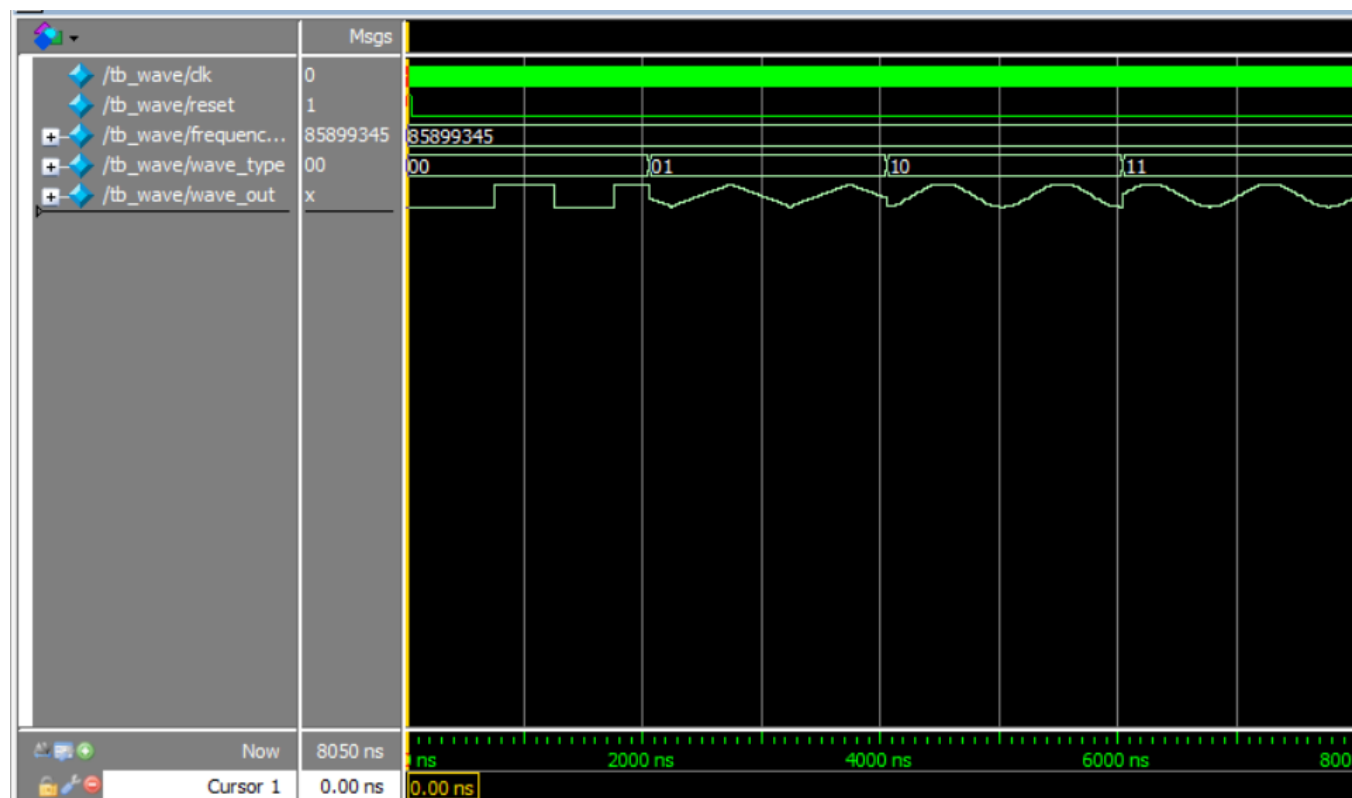
=> Kết luận: Thiết kế của nhóm tối ưu hơn về phần tài nguyên rất nhiều. Và thiết kế của nhóm cũng chạy tốt ở mức 100MHz được thiết lập trong bài báo



6. Mô phỏng và Nạp KIT DE2

6.1. Mô phỏng ModelSim

- Kết quả mô phỏng ModelSim cho thấy mạch thực hiện tạo sóng như mong đợi.
- Frequency step được cấu hình ở giá trị 85899345 tương đương với tần số sóng 1MHz.

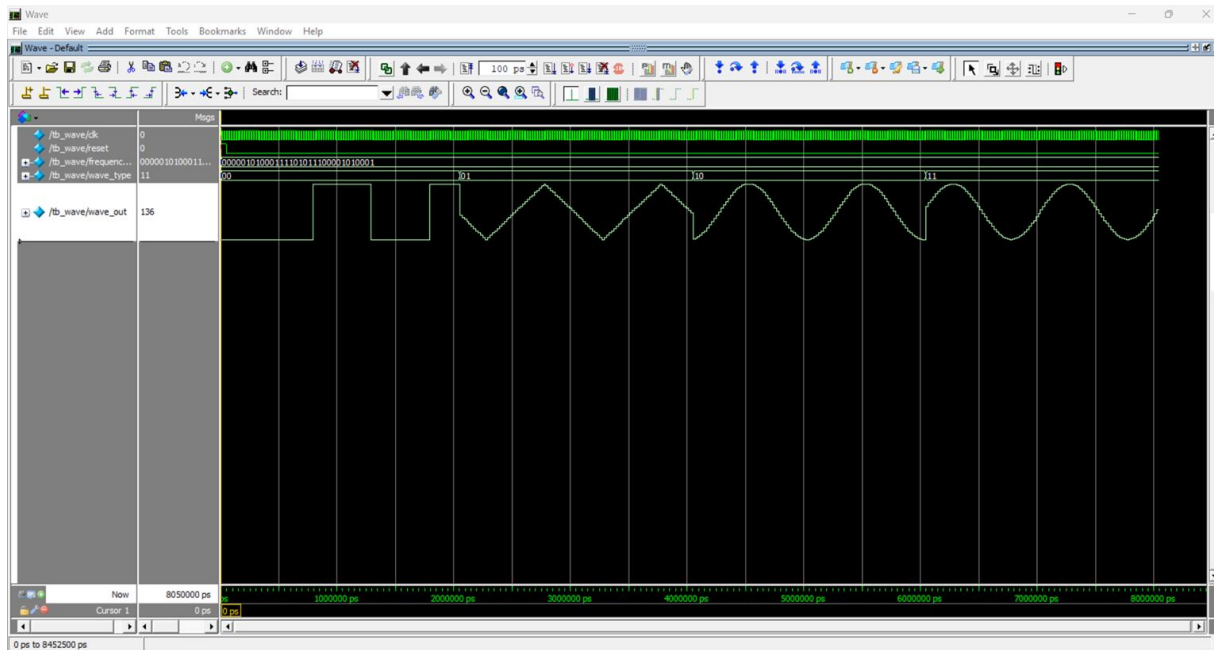




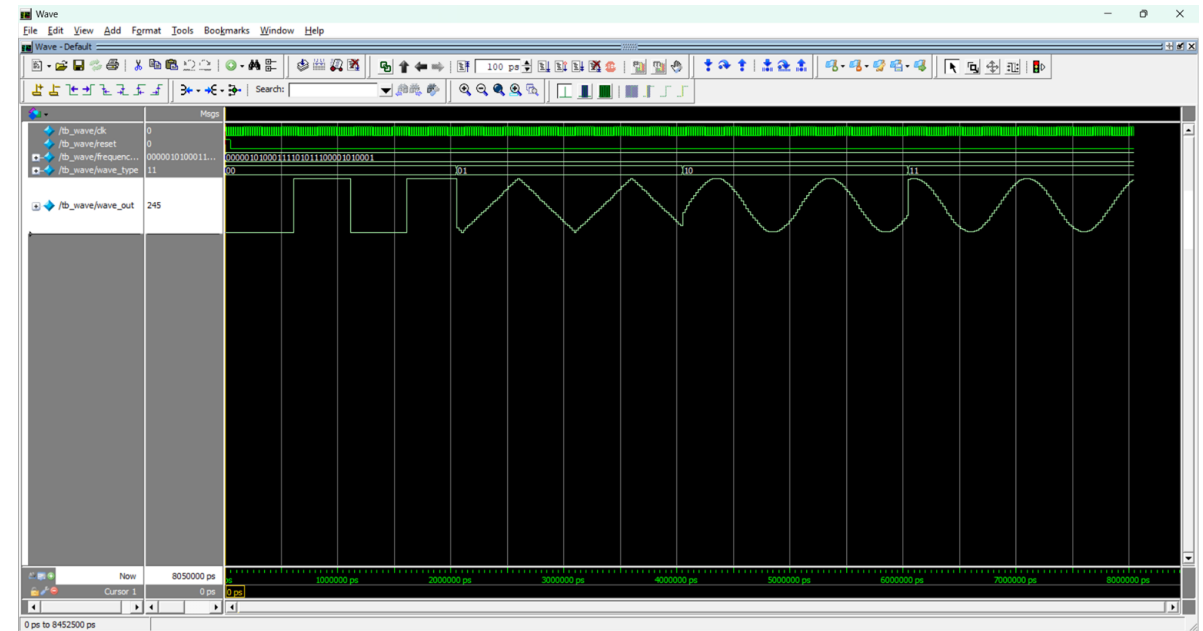
6. Mô phỏng và Nạp KIT DE2

6.1. Mô phỏng ModelSim

CORDIC



LUT





6. Mô phỏng và Nạp KIT DE2

6.2. Nạp KIT DE2

Triển khai mã C lên
phần cứng

```
#define PHASE_ACC_MAX 4294967296.0

void wave_set_frequency(float freq_MHz) {
    double step_calc = (freq_MHz * PHASE_ACC_MAX) / SYSTEM_CLOCK_MHz;
    uint32_t step = (uint32_t)step_calc;
    IOWR(WAVE_GEN_AVALON_0_BASE, REG_FREQ_OFFSET, step);
}

void wave_set_type(uint8_t type) {
    IOWR(WAVE_GEN_AVALON_0_BASE, REG_TYPE_OFFSET, (type & 0x03));
}

int main() {
    printf("Bat dau chay SignalTap Test...\n");
    wave_set_frequency(0.5);

    while (1) {
        printf("Dang phat: Song Vuong (500 KHz)\n");
        wave_set_type(WAVE_SQUARE);
        usleep(2000000);

        printf("Dang phat: Song Tam Giac (500 KHz)\n");
        wave_set_type(WAVE_TRIANGLE);
        usleep(2000000);

        printf("Dang phat: Song Sin (500 KHz)\n");
        wave_set_type(WAVE_SINE);
        usleep(2000000);

        printf("Dang phat: Song Cosine (500 KHz)\n");
        wave_set_type(WAVE_COSINE);
        usleep(2000000);
    }
}
```

Console Output:

```
wave_generator Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0
Dang phat: Song Tam Giac (500 KHz)
Dang phat: Song Sin (500 KHz)
Dang phat: Song Cosine (500 KHz)
Dang phat: Song Vuong (500 KHz)
Dang phat: Song Tam Giac (500 KHz)
```



6. Mô phỏng và Nạp KIT DE2

6.2. Nạp KIT DE2

Kiểm chứng dạng sóng thực tế trên chip bằng SignalTap II Logic Analyzer (Độ sâu: 1000 samples).





6. Mô phỏng và Nạp KIT DE2

6.2. Nạp KIT DE2

